

## EXAMEN PARCIAL – CORTE 2

### Árboles Binarios de Búsqueda (BST)

*Implementación con menú interactivo en Java*

|                   |                     |                 |               |
|-------------------|---------------------|-----------------|---------------|
| <b>Asignatura</b> | Estructura de Datos | <b>Corte</b>    | Segundo (25%) |
| <b>Programa</b>   | Ing. de Sistemas    | <b>Semestre</b> | 2026 – 1      |
| <b>Modalidad</b>  | Individual          | <b>Duración</b> | 120 minutos   |
| <b>Tipo</b>       | Teórico-práctico    | <b>Valor</b>    | 5.0 (cinco)   |

#### Identificación del estudiante

|                            |  |
|----------------------------|--|
| <b>Nombres y apellidos</b> |  |
| <b>Código / Documento</b>  |  |
| <b>Fecha de aplicación</b> |  |
| <b>Firma</b>               |  |

#### Competencia evaluada

Diseña e implementa estructuras de datos no lineales en Java, aplicando los principios del árbol binario de búsqueda (BST), sus algoritmos de inserción, búsqueda, modificación y recorridos, para resolver problemas computacionales en contextos del entorno productivo casanareño.

#### Resultados de aprendizaje

- Construye un BST a partir de un conjunto de datos respetando la propiedad de ordenamiento.
- Implementa al menos un recorrido (inorden, preorden o postorden) sobre un BST.
- Diseña un menú interactivo en consola que permita insertar, modificar y consultar los datos de un nodo.
- Analiza la complejidad temporal de las operaciones sobre el árbol.
- Argumenta la utilidad del BST frente a estructuras lineales en problemas reales.

### Indicaciones generales

Lea cuidadosamente cada punto antes de responder. El examen consta de cuatro (4) componentes: conceptual, trazado a mano, codificación en Java y análisis crítico. Puede usar lapicero negro o azul; los diagramas pueden hacerse a lápiz. Se permite calculadora simple. NO se permite material digital ni comunicación con compañeros. Las respuestas deben ser claras, ordenadas y argumentadas. Cualquier intento de fraude académico anula automáticamente la evaluación, conforme al Reglamento Estudiantil de Uniremington.

### Distribución de la calificación

| Parte | Componente                                             | Valor | Calificación |
|-------|--------------------------------------------------------|-------|--------------|
| I     | Fundamentos conceptuales (selección y respuesta corta) | 1.0   |              |
| II    | Trazado manual del BST + recorrido                     | 1.5   |              |
| III   | Codificación en Java (clase Nodo, BST y menú)          | 2.0   |              |
| IV    | Análisis crítico y aplicación al contexto              | 0.5   |              |
| TOTAL |                                                        | 5.0   |              |

## Contexto del examen

### Caso integrador: GanaderosCasanare S.A.S.

La empresa GanaderosCasanare S.A.S., ubicada en el corregimiento de Morichal (Yopal), administra el registro de sus reses mediante un código numérico único asignado a cada animal (chapeta electrónica). Hasta ahora utilizan un arreglo en Excel que ha crecido a más de 8.000 registros, y las búsquedas se han vuelto lentas e ineficientes. El gerente solicita migrar el manejo del inventario a una estructura de árbol binario de búsqueda (BST) implementada en Java, donde la clave de ordenamiento sea el código de la chapeta y cada nodo guarde además los datos del animal (raza, peso en kg, edad en meses). Usted, como ingeniero(a) en formación, debe demostrar que domina la construcción, recorrido y manipulación de un BST que resuelva esta necesidad.

### Datos de trabajo (úselos a lo largo del examen)

Conjunto de códigos de chapeta que llegan en este orden para ser insertados en el BST:

50, 30, 70, 20, 40, 60, 80, 35, 45, 75

## PARTE I. Fundamentos conceptuales (1.0 punto)

*Responda con claridad. Cada pregunta de selección vale 0.1 y cada respuesta corta vale 0.2.*

### 1.1 Selección única (marque con una X)

a) La propiedad fundamental de un árbol binario de búsqueda (BST) establece que, para cualquier nodo N:

- ☐ Todos los descendientes a la izquierda son mayores que N y los de la derecha son menores.
- ☒ Todos los descendientes a la izquierda son menores que N y los de la derecha son mayores.
- ☐ Los descendientes a la izquierda y a la derecha son iguales a N.
- ☐ El árbol siempre debe estar balanceado para considerarse BST.

b) El recorrido inorden de un BST produce los elementos:

- ☐ En el orden en que fueron insertados.
- ☒ En orden ascendente.
- ☐ En orden descendente.
- ☐ En orden aleatorio.

c) La complejidad promedio de la operación de búsqueda en un BST equilibrado es:

- ☐  $O(1)$
- ☐  $O(n)$

- (X)  $O(\log n)$
- ( )  $O(n^2)$

**d) Si se inserta una secuencia ya ordenada (10, 20, 30, 40, 50) en un BST vacío, la estructura resultante:**

- ( ) Será un árbol perfectamente balanceado.
- (X) Degenerará en una lista enlazada (peor caso).
- ( ) Será un árbol AVL automáticamente.
- ( ) No es posible insertarla en un BST.

## 1.2 Respuesta corta

**e) Defina con sus propias palabras qué es un nodo y qué atributos mínimos debe contener para representar un animal en el caso de GanaderosCasanare S.A.S.**

- Los nodos son los elementos que se guardan dentro del árbol ósea que en este caso cada nodo es el que representa cada animal del inventario ganadero y los atributos mínimos la raza el peso la edad y las referencias al hijo izquierda y derecha

**f) Explique la diferencia entre los recorridos preorden, inorden y postorden, indicando para qué problema típico se usa cada uno.**

- Preorden: en este primero se visita la raíz luego va hacia izquierda y después a la derecha y principal mente se usa para copiar o reconstruir arboles
- Inoden: en este primero se va de izquierda luego a la raíz y después a la derecha este muestra datos de forma ascendente
- Postorden: en este primero izquierda luego a la derecha y al final a la raíz este se normalmente para eliminar nodos o liberar memoria

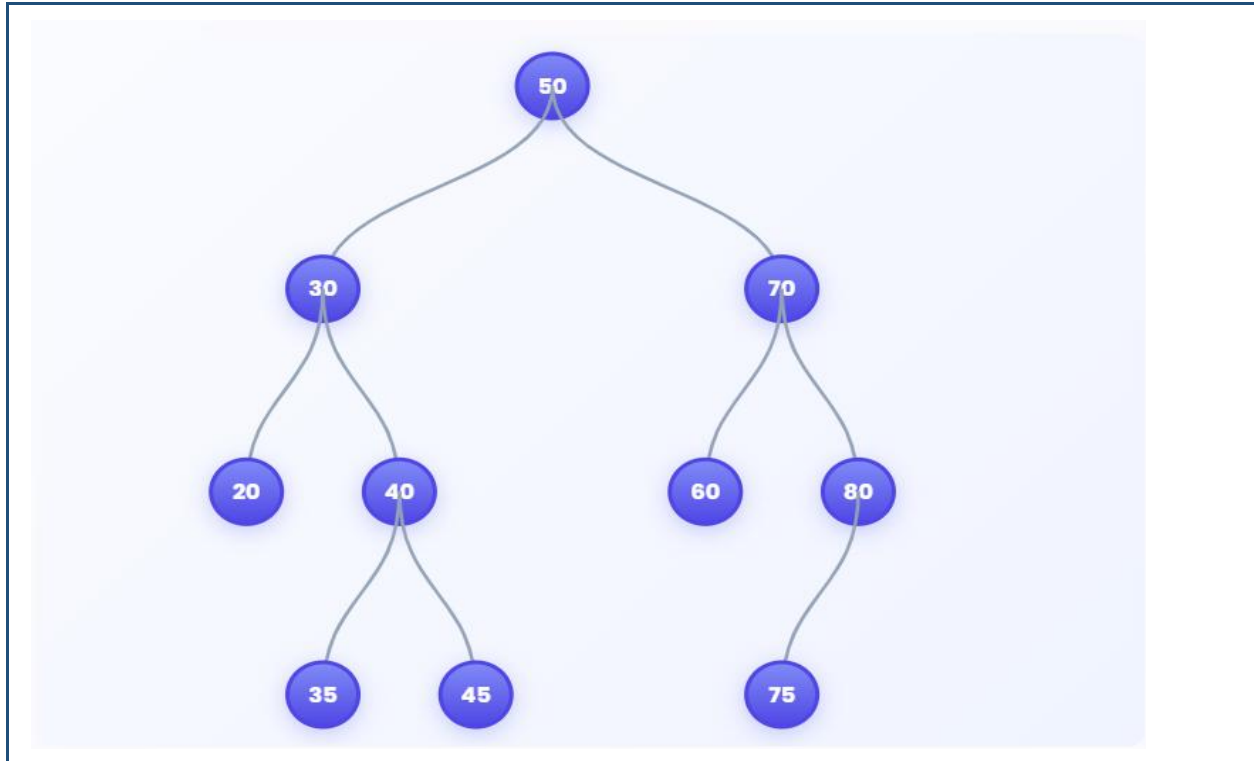
## PARTE II. Trazado manual del BST (1.5 puntos)

A partir de la secuencia de inserción definida en el contexto:

50, 30, 70, 20, 40, 60, 80, 35, 45, 75

### 2.1 Dibuje el árbol BST resultante (0.6)

Use el espacio en blanco. Indique claramente cada nodo, sus hijos izquierdo y derecho. Numere los nodos en el orden en que se insertaron.



### 2.2 Escriba los tres recorridos sobre el árbol anterior (0.6)

Inorden (Izquierda – Raíz – Derecha):

20, 30, 35, 40, 45, 50, 60, 70, 75, 80

Preorden (Raíz – Izquierda – Derecha):

50, 30, 20, 40, 35, 45, 70, 60, 80, 75

Postorden (Izquierda – Derecha – Raíz):

20, 35, 45, 40, 30, 60, 75, 80, 70, 50

### 2.3 Operaciones adicionales (0.3)

a) Suponga ahora que llega un nuevo registro con código 38. Dibuje el árbol resultante y señale en él la trayectoria de comparaciones que se realizan hasta encontrar la posición de inserción.

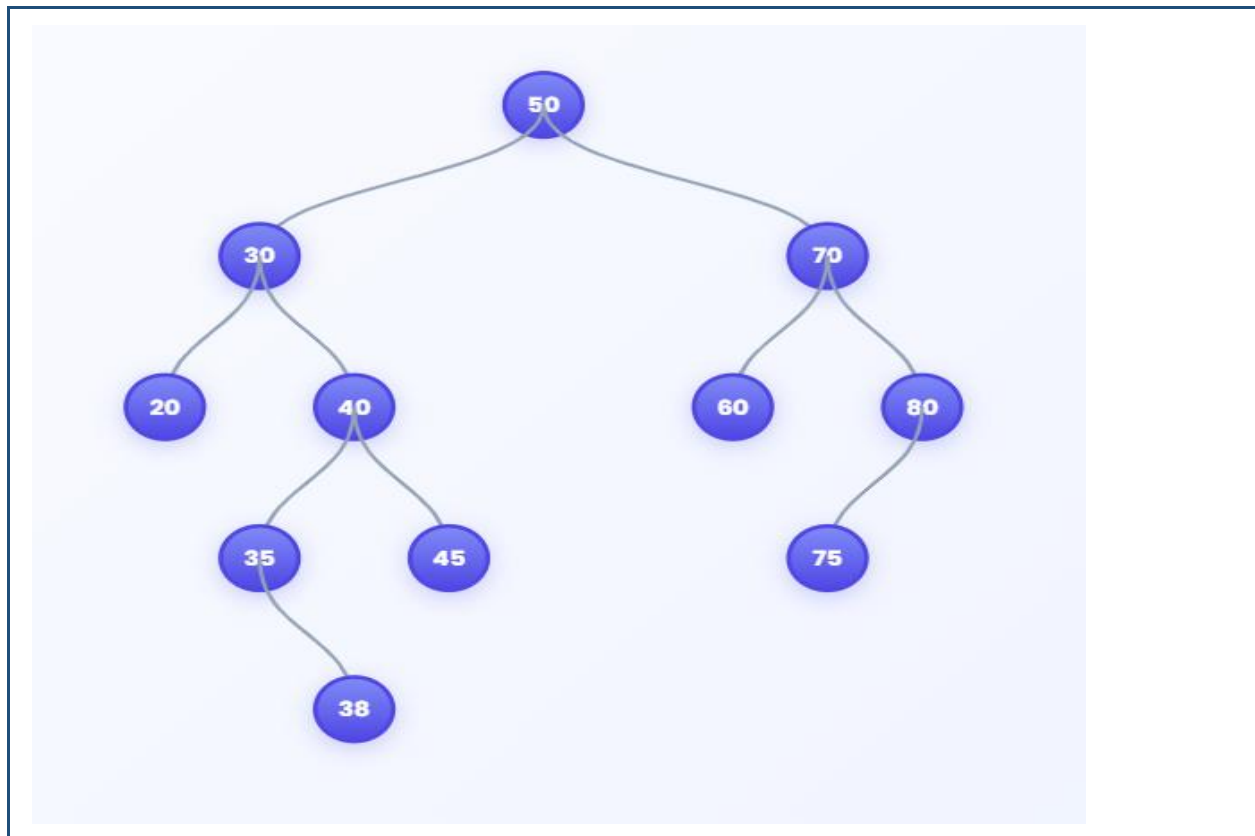
$38 < 50$

$38 > 30$

$38 < 40$

$38 > 35$

Son 4 comparaciones para llegar a 38 quedando a la derecha de 35



b) Indique la altura del árbol original (sin el 38), el número total de nodos y cuántas hojas posee.

Su altura es de 4 niveles tiene un total de 10 nodos y posee un total de 5 hojas las cuales son 20,35,45,60 y 75

## PARTE III. Codificación en Java (2.0 puntos)

*A continuación deberá implementar en lenguaje Java una solución completa que cumpla con los requerimientos del caso. Escriba el código a mano respetando la sintaxis. La rigurosidad sintáctica vale; los pequeños errores menores no penalizarán siempre que la lógica sea correcta.*

### 3.1 Clase Nodo (0.4)

Escriba la clase Nodo (también llamada NodoAnimal) que represente cada res del inventario. Debe contener: código de chapeta (int), raza (String), peso en kg (double), edad en meses (int), y las referencias a los hijos izquierdo y derecho. Incluya constructor y getters/setters.

```
// Archivo: NodoAnimal.java
public class NodoAnimal {
    // === Atributos ===
    private int codigo;
    private String raza;
    private double peso;
    private int edad;

    private NodoAnimal izquierdo;
    private NodoAnimal derecho;

    // === Constructor ===
    public NodoAnimal(int codigo, String raza,
                      double peso, int edad) {

        this.codigo = codigo;
        this.raza = raza;
        this.peso = peso;
        this.edad = edad;

        this.izquierdo = null;
        this.derecho = null;
    }

    // === Getters y Setters ===
    public int getCodigo() {
        return codigo;
    }

    public void setCodigo(int codigo) {
        this.codigo = codigo;
    }

    public String getRaza() {
        return raza;
    }
}
```

```
public void setRaza(String raza) {
    this.raza = raza;
}

public double getPeso() {
    return peso;
}

public void setPeso(double peso) {
    this.peso = peso;
}

public int getEdad() {
    return edad;
}

public void setEdad(int edad) {
    this.edad = edad;
}

public NodoAnimal getIzquierdo() {
    return izquierdo;
}

public void setIzquierdo(NodoAnimal izquierdo) {
    this.izquierdo = izquierdo;
}

public NodoAnimal getDerecho() {
    return derecho;
}

public void setDerecho(NodoAnimal derecho) {
    this.derecho = derecho;
}
}
```



### 3.2 Clase ArbolBST con inserción y un recorrido (0.7)

Implemente la clase ArbolBST con: el atributo raíz, el método insertar(int codigo, String raza, double peso, int edad) usando recursividad, y el método recorrerInorden() que imprima por consola los datos completos de cada animal en orden ascendente de código.

```
// Archivo: ArbolBST.java
public class ArbolBST {
    private NodoAnimal raiz;

    public ArbolBST() { this.raiz = null; }

    // Método público de inserción
    public void insertar(int codigo, String raza, double peso, int edad) {
        NodoAnimal nuevo =
            new NodoAnimal(codigo, raza, peso, edad);

        raiz = insertarRec(raiz, nuevo);
    }

    // Método privado recursivo de inserción
    private NodoAnimal insertarRec(NodoAnimal nodo, NodoAnimal nuevo) {
        if (nodo == null) {
            return nuevo;
        }

        if (nuevo.getCodigo() < nodo.getCodigo()) {
            nodo.setIzquierdo(
                insertarRec(nodo.getIzquierdo(), nuevo));
        } else if (nuevo.getCodigo() > nodo.getCodigo()) {
            nodo.setDerecho(
                insertarRec(nodo.getDerecho(), nuevo));
        }

        return nodo;
    }

    // Recorrido inorden
    public void recorrerInorden() {
        inordenRec(raiz);
    }
    private void inordenRec(NodoAnimal n) {
        if (n != null) {
```

```
        inordenRec(n.getIzquierdo());

        System.out.println(
            "Codigo: " + n.getCodigo() +
            " | Raza: " + n.getRaza() +
            " | Peso: " + n.getPeso() +
            " | Edad: " + n.getEdad());

        inordenRec(n.getDerecho());
    }
}
```

### 3.3 Métodos buscar(...) y modificar(...) (0.4)

Agregue a la clase ArbolBST dos métodos: buscar(int codigo) que retorne el NodoAnimal correspondiente o null si no existe, y modificar(int codigo, double nuevoPeso, int nuevaEdad) que actualice el peso y la edad del nodo si se encuentra, devolviendo true o false según el resultado.

```
// Búsqueda de un nodo por código de chapeta
public NodoAnimal buscar(int codigo) {
    while (actual != null) {

        if (codigo == actual.getCodigo()) {
            return actual;
        }

        if (codigo < actual.getCodigo()) {
            actual = actual.getIzquierdo();
        } else {
```

```
        actual = actual.getDerecho();
    }
}

return null;
}
}

// Modificación de los datos de un nodo existente
public boolean modificar(int codigo, double nuevoPeso, int nuevaEdad) {
    NodoAnimal encontrado = buscar(codigo);

    if (encontrado != null) {

        encontrado.setPeso(nuevoPeso);
        encontrado.setEdad(nuevaEdad);

        return true;
    }

    return false;
}
}
```

### 3.4 Clase Principal con menú interactivo (0.5)

Escriba la clase Principal que contenga el método main y muestre por consola, en bucle, el siguiente menú. Use Scanner para leer la opción y los datos.

```
=====
GanaderosCasanare S.A.S. - Inventario BST
=====
1. Insertar un nuevo animal
2. Modificar datos de un animal existente
3. Consultar datos de un animal (por código)
4. Listar inventario completo (recorrido inorden)
5. Salir
-----
Seleccione una opción: _
import java.util.Scanner;

public class Principal {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        ArbolBST arbol = new ArbolBST();

        int opcion;

        do {

            System.out.println("=====");
            System.out.println(" GanaderosCasanare S.A.S.");
            System.out.println("=====");
            System.out.println("1. Insertar animal");
            System.out.println("2. Modificar animal");
            System.out.println("3. Buscar animal");
            System.out.println("4. Mostrar inventario");
            System.out.println("5. Salir");
            System.out.print("Seleccione una opcion: ");

            opcion = sc.nextInt();

            switch(opcion) {
                case 1:
                    System.out.print("Codigo: ");
                    int codigo = sc.nextInt();

                    sc.nextLine();
```

```
System.out.print("Raza: ");
String raza = sc.nextLine();

System.out.print("Peso: ");
double peso = sc.nextDouble();

System.out.print("Edad: ");
int edad = sc.nextInt();

arbol.insertar(codigo, raza, peso, edad);

System.out.println("Animal insertado");
break;
case 2:
    System.out.print("Codigo: ");
    int codMod = sc.nextInt();

    System.out.print("Nuevo peso: ");
    double nuevoPeso = sc.nextDouble();

    System.out.print("Nueva edad: ");
    int nuevaEdad = sc.nextInt();

    boolean modificado =
        arbol.modificar(codMod,
                        nuevoPeso,
                        nuevaEdad);
    if(modificado) {
        System.out.println("Datos modificados");
    } else {
        System.out.println("Animal no encontrado");
    }
    break;
case 3:
    System.out.print("Codigo a buscar: ");
    int codBuscar = sc.nextInt();
    NodoAnimal encontrado =
        arbol.buscar(codBuscar);
    if(encontrado != null) {
        System.out.println(
            "Raza: " + encontrado.getRaza());
    } else {
        System.out.println("No existe");
    }
}
```

```
        break;
    case 4:
        arbol.recorrerInorden();
        break;
    case 5:
        System.out.println("Programa finalizado");
        break;
    default:
        System.out.println("Opcion invalida");
    }
} while(opcion != 5);
sc.close();
}
}
```

Escriba el método main implementando el bucle del menú con un switch que invoque los métodos de ArbolBST. La opción 5 debe finalizar el programa de forma controlada.

## PARTE IV. Análisis crítico y aplicación (0.5 puntos)

*Responda de manera argumentada. La calidad de la justificación es lo que se evalúa, no la extensión.*

### 4.1 Pregunta de análisis (0.3)

El gerente de GanaderosCasanare S.A.S. le comenta: «Si los códigos de chapeta los entregan en orden ascendente al momento del registro (1001, 1002, 1003, ...), el BST seguirá siendo eficiente». ¿Está usted de acuerdo? Justifique técnicamente su respuesta indicando qué le ocurriría al árbol y qué estrategia propondría para resolver el problema.

- No estoy de acuerdo porque si los códigos se guardan de en orden ascendente el árbol se va a empezar a llenar solo hacia un lado y no quedaría balanceado y eso aria que cuando se valla a buscar algún dato sea mas lento porque tocaría revisar casi todos los nodos de uno en uno

### 4.2 Aplicación al entorno (0.2)

Mencione otros dos (2) escenarios del contexto casanareño (sector petrolero, agroindustrial, educativo o de servicios públicos en Yopal) donde sería pertinente utilizar un BST en lugar de una lista o un arreglo. Justifique brevemente cada caso.

- En colegios se podría usar para guardar los códigos de los estudiantes ayudando a buscar las notas o sus datos de forma más rápida
- En una empresa de servicio público se podría usar para guardar los números del contrato de los usuarios y encontrar mucho mas rápido la información de sus pagos

Rúbrica de evaluación

La rúbrica orienta tanto al estudiante como al docente. Cada criterio se valora en una escala de cuatro niveles.

| Criterio                     | Superior (1.0)                                          | Alto (0.75)                                     | Básico (0.50)                                            | Bajo (0.25)                                            |
|------------------------------|---------------------------------------------------------|-------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| Trazado del BST y recorridos | Árbol correcto y los 3 recorridos exactos.              | Árbol correcto y 2 recorridos exactos.          | Árbol con un error leve y al menos 1 recorrido correcto. | Árbol con errores estructurales o recorridos ausentes. |
| Clase Nodo y BST en Java     | Sintaxis correcta, atributos y métodos completos.       | Implementación funcional con errores menores.   | Implementación parcial; falta un método clave.           | No compila o presenta errores graves de lógica.        |
| Menú interactivo             | Menú completo, robusto y con manejo de la opción salir. | Menú funcional pero sin validación de entradas. | Menú incompleto (faltan opciones).                       | Menú ausente o no funcional.                           |
| Análisis crítico             | Argumento técnico sólido con propuesta de mejora.       | Argumento correcto pero sin propuesta clara.    | Argumento superficial.                                   | Respuesta ausente o incorrecta.                        |


**LINK GITHUB DE APLICACIÓN:**  
Espacio reservado para LINK de GitHub de la aplicación.

<https://github.com/kyliandoncel85-code/PARCIAL-2-BST>

|                                        |                                 |
|----------------------------------------|---------------------------------|
| <b>Calificación final:</b> _____ / 5.0 | <b>Firma del docente:</b> _____ |
|----------------------------------------|---------------------------------|